Experiment 2: Sparse Matrix Operations

Problem Statement:
WACP for sparse matrix realization and operations on it- Simple Transpose and Fast Transpose.

Objectives:
- To study the concept of sparse matrix, how it is stored and displayed.
- To understand the implementation of sparse matrix operations- Simple Transpose and Fast Transpose.

Implementation:

- Platform:
i. 64-bit Open Source Linux or its derivatives.
ii. Open Source C Programming tool like gcc/Eclipse editor.

- Pseudo code:

```
Algorithm compact(A,m,n,B)
{// m and n are total nos of rows and cols of original matrix.
B(0,0) = m;
B(0,1) = n;
K=1;

for i=0 to m
{
```

```
for j=0 to n
{
   if (A(i,j)!=0)
     {
        B(k,0) = i;
        B(k,1) = j;
        B(k,2) = A(i,j);
        k++;
     } // end of for j
  }     // end for i
}
B(0,2) = k-1;
}          // end compact.
```

Algorithm SimpleTranspose (A, B)
// A is a matrix represented in sparse form. B is set to be its transpose.

```
(m, n, t) := (A(0,0), A(0,1), A(0,2))
(B(0,0), B(0,1), B(0,2)) := (n, m, t)
if t<=0 then return                    //check for zero matrix
        q:=1                           // q is position of next term in B
        for col:= 0 to n do // transpose by column
           for p:=1 to t do // for all non-zero term do
              if A(p,1) = col then // correct column
                {
                 (B(q,0), B(q,1), B(q,2)):= (A(p,1), A(p,0), A(p,2))
                 q:=q+1
```

Sundaram
FOR EDUCATIONAL USE

```
            }
          }
        end
      end
end SimpleTranspose

Algorithm FastTranspose(A,B)
// A is a matrix represented in sparse form. B is set to be its
 transpose. t is non-zero terms.
 declare S(n), T(n);                    //local array

 info(m,n,t) := (A(0,0),A(0,1),A(0,2))
 (B(0,0), B(0,1), B(0,2)) := (n,m,t)
 if t <= 0 then return                  //check for zero matrix
    for i := 0 to n do
        S(i) := 0                       //S(k) is the nos of elements
        end                                 in row k of B
    for i := 1 to t do
        S(A(i,1)) := S(A(i,1))+1        //elements of row k of B
        end

T(0) := 1                               //T(i) is the starting position of i in B
for i := 1 to n do
    T(i) := (T(i-1)) + (S(i-1))
end

for i := 1 to t do      // move all the elements of A to B.
    j := A(i,1)         // j is the row in B
  (B(T(j),0), B(T(j),1), B(T(j),2)) := (A(i,1), A(i,0), A(i,2))
```

```
        T(j)++
    end
end Fast Transpose.

Function main (){
    Display menu :
        1. Compact form
        2. Simple Transpose
        3. Fast Transpose
    Accept the dimensions of matrix and accept choice.

    Switch (ch) : {
    Case 1: Input A[row][cols]
            print "Enter nos of rows (m) and columns (n):"
            read m,n
            print "Enter matrix elements of A:"
            for i := 0 to m-1 do
                for j := 0 to n-1 do
                    read A(i,j)
                end
            end

            compact (A,m,n,B)
            print "Compact form created in matrix C."
            break
    Case 2: Simple Transpose (C,B)
            print "Simple Transpose complete in matrix B"
            break
```

Sundaram®
FOR EDUCATIONAL USE

```
        Case 3: FastTranspose (C, B)
                print "Fast Transpose complete in matrix B"
                break
        default: print "Invalid choice!"
                break
        }
}
```

- Time Complexity:

1. Simple Transpose = $O(n * t)$
2. Fast Transpose = $O(n + t)$

Conclusion:

Thus, implemented sparse matrix operations assignment. This system is able to perform different operations on sparse matrices such as simple and fast transpose and their time complexities.

FAQs:

1. What is a sparse matrix? List the applications.

Ans. A sparse matrix is defined as any $m \times n$ matrix that contains a large number of zeroes. Instead of storing every element, which would be inefficient for memory, a sparse matrix is represented in a compact form.

Characteristics and Representation:

a. Triplet Form: A sparse matrix represents non-zero elements using <row, column, value> triplets.

b. Compact storage: The first row stores metadata followed by rows

of triplets for each non-zero element.

c. Efficiency: Storing only non-zero values significantly reduces memory usage compared to a full 2D array.

d. Operations: Key algorithms on compact sparse matrices include Addition, Simple Transpose, and Fast Transpose.

Applications of Sparse Matrices

i. Scientific Computing

ii. Machine Learning and NLP

iii. Graph Theory

iv. Image Processing

2. Represent sparse matrices with suitable data structures? Explain with an example simple and fast transpose?

Ans. Sparse matrices are represented using a compact form or triplet representation, which significantly reduces storage requirements compared to standard matrix. Instead of storing all elements, including zeroes, only the non-zero elements are recorded in an array of triplets.

i. Triplet Representation:

a. Data Structure: Stores non-zero elements as <row, col, value> triplets in a $(t+1) \times 3$ matrix for t non-zero entries.

b. Header Row: Row 0 stores matrix metadata: [total-rows, total-cols, total-non-zero].

c. Efficiency: Saves memory by storing only non-zero values.

ii. Simple Transpose:

a. Mechanism: Scans the matrix n times (once per column) to pull non-zero entries in column order, swapping row and column indices.

Sundaram®

FOR EDUCATIONAL USE

b. Time Complexity: $O(n \times t)$, where n = columns and t = non-zero terms

Eg.

A	Col 1	Col 2	Col 3
Row 0	6	6	8
Row 1	0	0	15
Row 2	0	3	22
Row 3	0	5	-15
Row 4	1	1	11
Row 5	1	2	3
Row 6	2	3	-6
Row 7	4	0	91
Row 8	5	2	28

Compact form of matrix

A	Col 0	Col 1	Col 2
Row 0	6	6	8
Row 1	0	0	15
Row 2	0	4	91
Row 3	1	1	11
Row 4	2	1	3
Row 5	2	5	28
Row 6	3	0	22
Row 7	3	2	-6
Row 8	3	0	-15

Single Transpose

c. Example logic:
- The algorithm scans the original matrix n times
- For each column index, it iterates through all non-zero elements to find matches.
- When a match is found, the row and column indices are swapped and added to the new matrix.

iii. Fast Transpose:

a. Mechanism: Transposes in a single pass by pre-computing exact output positions using two auxillary arrays.
- total [ ]: Counts non-zero elements per column
- index [ ]: Tracks starting positions for each row in the transposed matrix

b. Time Complexity: $O(n+t)$, making it significantly faster for large matrices.

c. Example logic:
- Traverses the original triplet matrix once to count how many elements exist in each column.
- Use counts in total[] to fill index[]
- Traverse the original matrix one last time, For each element, look up its target location using index[col-no], place it there, and increment the index for that column.

3. Find out the addition of two sparse matrices in triplet form and also Simple and Fast Transpose.

M1 =

	4	5	6
1	0	3	5
2	1	3	8
3	1	4	45
4	2	3	4
5	3	2	45
6	4	1	2

(1)

M2 =

	4	5	6
1	0	3	7
2	0	4	6
3	1	4	4
4	2	1	8
5	3	2	45
6	4	4	21

(2)

Ans. Addition of sparse matrix:

Since, (M1(0,0), M1(0,1)) = (M2(0,0), M2(0,1))

M = M1 + M2 =

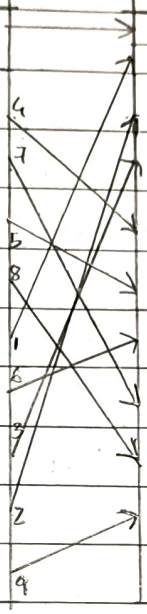
	4	5	8
1	0	3	12
2	0	4	6
3	1	3	8
4	1	4	49
5	2	1	8
6	2	3	4
7	3	2	90
8	4	1	2
9	4	4	21

	M1 i	M2 j	M k	
1	1	1	1	← initializ^n
2	2	2	2	
3	2	3	3	
4	3	3	4	
5	4	4	5	
6	4	5	6	
7	5	5	7	
8	6	6	8	
9	7×	6	9	

Sundaram
FOR EDUCATIONAL USE

Simple Transpose of matrix:

M	col 0	col 1	col 2
Row 0	4	5	8
Row 1	0	3	12
Row 2	0	4	6
Row 3	1	3	8
Row 4	1	4	49
Row 5	2	1	8
Row 6	2	3	4
Row 7	3	2	90
Row 8	4	1	2
Row 9	4	4	21



M_T	col 0	col 1	col 2
Row 0	4	5	8
Row 1	1	2	8
Row 2	1	4	2
Row 3	2	3	90
Row 4	3	0	12
Row 5	3	1	8
Row 6	3	2	4
Row 7	4	0	6
Row 8	4	1	49
Row 8	4	4	21
Row 9	5	4	8

Fast Transpose of matrix:

M	col 0	col 1	col 2
Row 0	4	5	8
Row 1	0	3	12
Row 2	0	4	6
Row 3	1	3	8
Row 4	1	4	49
Row 5	2	1	8
Row 6	2	3	4
Row 7	3	2	90
Row 8	4	1	2
Row 9	4	4	21

total

0	2	1	3	3
0	1	2	3	4

index

1	1	3	4	7	10
0	1, 2, 3	2, 4	3, 5	4, 8, 9	5

location 3 4 3 4 1 3 → 2 1 4

M_T	col 0	col 1	col 2
Row 0	4	5	8
Row 1	1	2	8
Row 2	1	4	2
Row 3	2	3	90
Row 4	3	0	12
Row 5	3	1	8
Row 6	3	2	4
Row 7	4	0	6
Row 8	4	1	49
Row 9	4	4	21